Data Processing

STM32 Primer — Chapter 6

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. Shift Basics
5. The Barrel Shifter
6. Addressing Modes
7. Advanced Arithmetic Operations
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. Summary

Why This Chapter Matters

Every peripheral is controlled by setting and clearing single bits inside a 32-bit register, without changing neighbouring bits. Load, modify, store back: this is the most common pattern in embedded firmware. This chapter builds it from instructions. You will then know exactly what `GPIOA->MODER |= (1U << 10)` executes, and why.

After this chapter, you can:

1. Choose the right arithmetic instruction; explain why a 32-bit constant may need two.
2. Write a read-modify-write that spares neighbouring bits.
3. Predict shift results; tell logical from arithmetic shifts.
4. Use the flexible second operand and barrel shifter.
5. Use offset, pre-indexed and post-indexed addressing.
6. Use a scaled register offset to reach an array element.

© Muním Zabidi

By the end of this chapter you can read how the compiler turns this C into four instructions, and say what each one does to the bits:

```
@ GPIOA->MODER &= ~(3U << 10);  // clear bits 11:10
@ GPIOA->MODER |=  (1U << 10);  // write 01

LDR   R1, [R0]        @ read MODER
BIC   R1, R1, #0xC00  @ clear the field
ORR   R1, R1, #0x400  @ write 01
STR   R1, [R0]        @ write back
```

Questions this code raises, and where this chapter answers them:

Question raised	Answered in
What does `BIC` do?	Logical instructions
Where did `0xC00` come from?	Shifts and masks
Can the shift be free?	The barrel shifter
How are array elements reached?	Addressing modes

© Muním Zabidi

BEAMER EXTRA

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. Shift Basics
5. The Barrel Shifter
6. Addressing Modes
7. Advanced Arithmetic Operations
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. Summary

Arithmetic Instructions

Instruction	Operation	Note
`ADD`	`Rd = Rn + Operand2`	
`SUB`	`Rd = Rn − Operand2`	
`RSB`	`Rd = Operand2 − Rn`	Negation: `RSB R0, R1, #0`
`MUL`	`Rd = Rn × Rm`	Single cycle on the M4
`SDIV` / `UDIV`	`Rd = Rn ÷ Rm`	2–12 cycles

Loading a 32-bit constant

The immediate value must fit *inside* the instruction, which is itself only 32 bits. So a full constant needs either `MOVW` then `MOVT` (two instructions, no memory access), or `LDR Rn, =value` (one instruction plus a load from a literal pool).

Arithmetic in C and in Assembly Load–process–store, once more

In C

```
total = a + b - c;
```

In assembly

```
LDR   R0, =a
LDR   R0, [R0]    @ R0 = a
LDR   R1, =b
LDR   R1, [R1]    @ R1 = b
LDR   R2, =c
LDR   R2, [R2]    @ R2 = c
ADD   R3, R0, R1  @ R3 = a + b
SUB   R3, R3, R2  @ R3 = (a+b) - c
LDR   R4, =total
STR   R3, [R4]
```

One C statement, one register-only computation

Every operand is loaded into a register before either `ADD` or `SUB` can touch it. Neither instruction ever reads or writes memory directly — the result is stored only once, at the very end.

The compiler is free to reorder or reuse registers, but the shape never changes: load every operand, compute in registers, store the result.

Table of Contents

1 Why This Matters

2 Arithmetic Instructions

3 **Logical Instructions and Bit Manipulation**

4 Shift Basics

5 The Barrel Shifter

6 Addressing Modes

7 Advanced Arithmetic Operations

8 Optional: How a Compiler Chooses Between Shift and Multiply

9 Summary

Logical Instructions

Instruction	Operation	What it is for
`AND`	`Rd = Rn & Op2`	**Masking** — keep only bits you care about
`ORR`	`Rd = Rn | Op2`	**Setting** bits to 1
`EOR`	`Rd = Rn ^ Op2`	**Toggling** bits
`BIC`	`Rd = Rn & ~Op2`	**Clearing** bits to 0
`ORN`	`Rd = Rn | ~Op2`	Setting bits where the mask is **0**

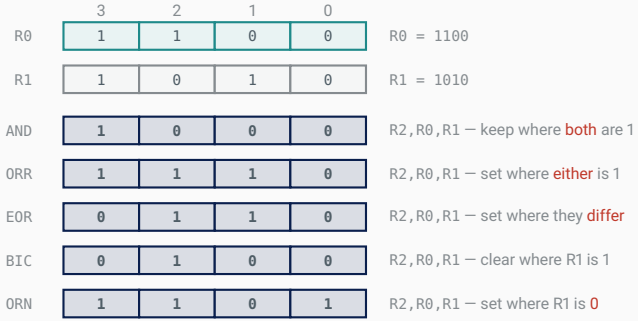
`BIC` is the one without a matching C operator

In C you clear bits with `x &= ~mask`. ARM gives this its own instruction. So the compiler emits *one* `BIC`, where you might expect `MVN` followed by `AND`. `ORN` does the same for `x |= ~mask`.

© Muním Zabidi

The Five Operations on the Same Bits

R0 contains 12_{10}, R1 contains 10_{10}.



© Muním Zabidi

Read–Modify–Write The sequence behind every peripheral write

In C

```
GPIOA->MODER &= ~(3U << 10);

GPIOA->MODER |=  (1U << 10);
```

In assembly

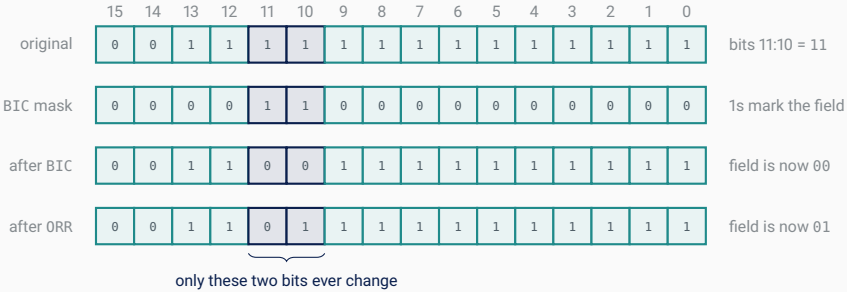
```
LDR   R3, =0x40020000
LDR   R0, [R3]
BIC   R0, R0, #0xC00  @ clear bits 11:10
ORR   R0, R0, #0x400  @ set bit 10
STR   R0, [R3]
```

Why you must keep the neighbouring bits

This register holds settings for sixteen pins. Writing the whole register would reconfigure every one of them. Load it, change only your bits, then store it back.

`3U << 10 = 0xC00;  1U << 10 = 0x400`. This exact sequence sets up PA5 as an output.

Following the Bits Through Clear-Then-Set



`ORR` alone cannot turn `11` into `01` — OR can only turn bits on. `BIC` must clear the field first.

Assembly Step by C Step

Step	Assembly	Equivalent C
Load the register	`LDR R1, [R0]`	implicit in `GPIOA->MODER`
Clear the field	`BIC R1, R1, R2`	`&= ~(3U << 10)`
Set the new value	`ORR R1, R1, #0x400`	`|= (1U << 10)`
Store the register	`STR R1, [R0]`	implicit in the `->` assignment

Two rows are “implicit”

C never issues a separate load or store on its own — the compiler emits `LDR`/`STR` automatically around every `->MODER` reference. Only the middle two rows have a direct C spelling.

© Muním Zabidi

Read-Modify-Write at Five Levels

Level	Representation
Physical hardware	Two configuration flip-flops selecting the PA5 output driver
Register	`GPIOA_MODER` bits 11:10
C	`MODER &= ~(3U << 10); MODER |= (1U << 10);`
Assembly	`LDR` / `BIC` / `ORR` / `STR`
Machine code	Four Thumb-2 encodings

One operation, five simultaneous descriptions

Read downward: a flip-flop is a bit, a bit is in a register, a register has a name in C, a name becomes four instructions, four instructions become bytes in Flash. Read upward: the C statement reaches a flip-flop.

© Munim Zabidi

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. **Shift Basics**
5. The Barrel Shifter
6. Addressing Modes
7. Advanced Arithmetic Operations
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. Summary

The Four Shifts

Shift operations move all bits in a register left or right by a specified number of positions, useful for fast arithmetic and bit manipulation.

Connection to Digital Electronics

You have already encountered these concepts in Digital Electronics as hardware shift registers (SISO, SIPO, PISO, PIPO) and ring counters.

Instruction	Meaning	Effect
`LSL #n`	Logical shift left	Multiplies by 2^n; zeros fill in from the right
`LSR #n`	Logical shift right	Divides an unsigned value by 2^n; zeros fill in from the left
`ASR #n`	Arithmetic shift right	Divides a signed value by 2^n; the sign bit fills in from the left
`ROR #n`	Rotate right	Bits that fall off the right re-enter on the left

Only one bit decides LSR versus ASR

Both shift right. The difference is what fills the vacated bits: `LSR` always fills with 0. `ASR` copies the sign bit, so a negative number stays negative after the shift.

© Muním Zabidi

Shifts as Ordinary Instructions

```
LSL   R0, R1, #2  @ R0 = R1 << 2   (R1 times 4)
LSR   R0, R1, #1  @ R0 = R1 >> 1   (R1 divided by 2, unsigned)
ASR   R0, R1, #1  @ R0 = R1 >> 1   (R1 divided by 2, signed)
```

Written this way, a shift is a complete instruction: it reads `R1`, shifts it, and writes the result to `R0` — exactly like `ADD` or `MOV`.

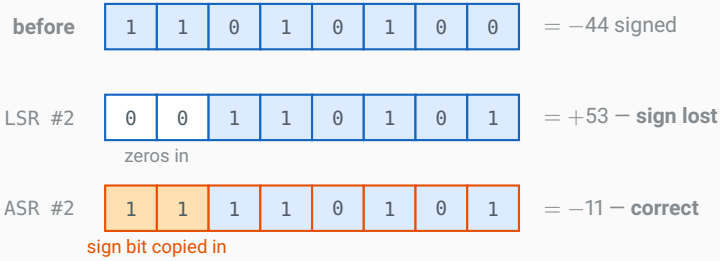
The idea worth carrying forward

A shift is useful by itself. But ARM can also feed a shifted value directly into another instruction, without a separate shift step first. That is the subject of the next section.

© Muním Zabidi

Logical vs. Arithmetic Right Shifts

Both shift right, and both look the same for positive numbers. The difference shows only when the sign bit is 1. Here `11010100` (-44) is shifted right by two:



The bits shifted *out* are the same. Only the bits shifted *in* differ. LSR suits masks and unsigned values. ASR divides a signed value by 2^n.

Predict the Shift Same bits, two different shifts

`R1 = 0xFFFFFFD4` (this is -44 as a signed value). Predict the result of each shift by 2:

```
LSR   R2, R1, #2
ASR   R3, R1, #2
```

Your prediction

- What is R2?
- What is R3?
- Which one is correct signed division?

Predict the Shift Same bits, two different shifts

`R1 = 0xFFFFFFD4` (this is −44 as a signed value). Predict the result of each shift by 2:

```
LSR   R2, R1, #2
ASR   R3, R1, #2
```

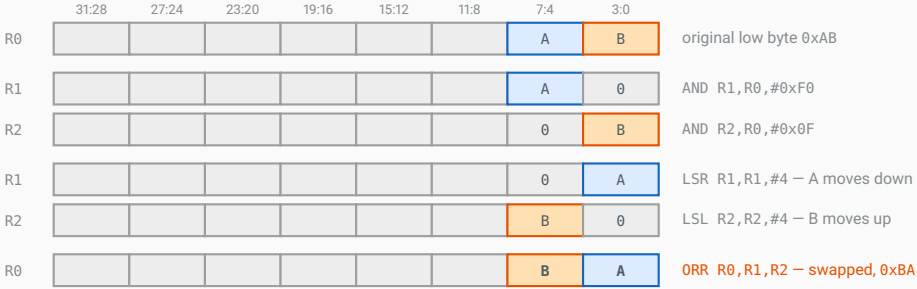
Your prediction

- What is R2?
- What is R3?
- Which one is correct signed division?

The answer

`R2 = 0x3FFFFFF5` (a large positive number: zeros fill in, the sign is lost). `R3 = 0xFFFFFFF5` (−11): the sign bit is copied in. ASR divides a signed value.

Swapping Two Nibbles Isolate with AND, recombine with ORR



Multiplying by a Power of Two Strength reduction: replacing multiply with shift

In C

```
total = 4 * a + b;
```

a and b are already in R0 and R1 — no LDR needed here.

In assembly

```
LSL   R2, R0, #2  @ R2 = a << 2
                  @    = a * 4
ADD   R3, R2, R1  @ R3 = (4*a) + b
```

Why the compiler prefers LSL over MUL

Multiplying by 4 is multiplying by 2^2. A left shift by 2 bits produces exactly that result, and on most cores a shift is cheaper than a general multiply. The compiler makes this substitution automatically — it is called strength reduction. No MUL instruction appears at all.

A power-of-two constant becomes one shift. Other constants can avoid MUL too, for example `5 * a` becomes one ADD with a shifted operand. See the optional slide on the compiler's choices.

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. Shift Basics
5. **The Barrel Shifter**
6. Addressing Modes
7. Advanced Arithmetic Operations
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. Summary

What is the Barrel Shifter?

A **barrel shifter** is a specialized digital circuit that can shift or rotate a data word by any number of bit positions in a single clock cycle.

Why is it a game-changer in ARM?

- **Hardware Efficiency:** Unlike a basic shift register (which shifts 1 bit per cycle), it uses combinational multiplexers to shift *n* bits instantly.
- **Zero-Cost Operations:** In ARM processors, it is wired directly into the ALU input path.
- **Combined Execution:** Shifts and arithmetic/logic operations happen in the same instruction cycle with zero clock penalty.

`ADD R0, R1, R2, LSL #2` ⟶ Shift `R2` left by 2, then add to `R1` in 1 cycle.

© Munim Zabidi

The Flexible Second Operand

```
Opcode{S}{cond}   Rd,   Rn,   Operand2
```

Form	Written	Meaning
Constant	`#constant`	An immediate encoded in the instruction
Register	`Rm`	A register holding the second operand
Register with shift	`Rm, LSL #n`	`Rm` shifted before use

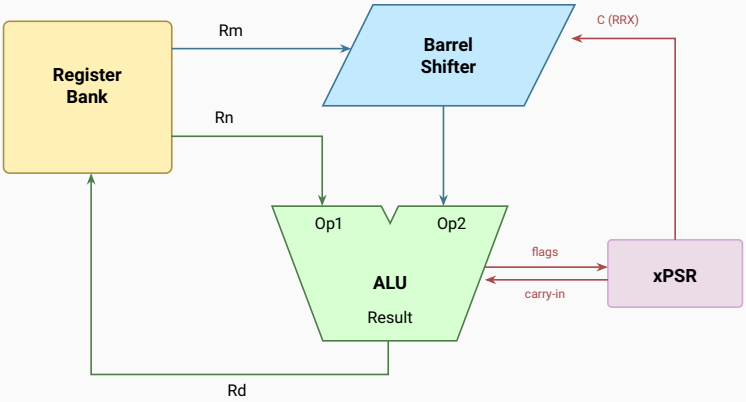
```
ADD   R0, R1, R2, LSR #3  @ R0 = R1 + (R2 >> 3)
ORR   R0, R1, R2, LSL #2  @ R0 = R1 | (R2 << 2)
```

Folded into the instruction, not a separate step

The barrel shifter sits in the datapath *before* the ALU, so the shift and the operation happen together: no separate shift instruction is needed. `Rm` itself stays unchanged — only the value entering the ALU is shifted.

© Muním Zabidi

Where the Barrel Shifter Sits



`Rm` feeds the barrel shifter. Its output enters the ALU as `Op2`. `Rn` feeds `Op1` directly. The result is written back to `Rd`, and the flags update in xPSR. The carry bit of xPSR is sent to ALU and barrel shifter.

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. Shift Basics
5. The Barrel Shifter
6. **Addressing Modes**
7. Advanced Arithmetic Operations
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. Summary

The Four Addressing Modes

Assume `R1` = `0x2000` before each instruction.

Instruction	Mode	Reads from	R1 afterwards
`LDR R0, [R1]`	register	`0x2000`	`0x2000`
`LDR R0, [R1, #8]`	offset	`0x2008`	`0x2000`
`LDR R0, [R1, #8]!`	pre-indexed	`0x2008`	`0x2008`
`LDR R0, [R1], #8`	post-indexed	`0x2000`	`0x2008`

The Writeback Indicator (`!`)

- The exclamation point `!` explicitly commands auto-update (writeback) for pre-indexed access.
- Post-indexed access **implicitly updates** the base register without requiring a `!` symbol.

1. Simple offset

```
LDR   R0, [R1, #8]
@ address = R1 + 8
@ R1 unchanged
```

2. Pre-indexed: Calculate first, then access.

```
LDR   R0, [R1, #8]!
@ address = R1 + 8
@ R1 = R1 + 8
```

3. Post-indexed: Access first, then calculate.

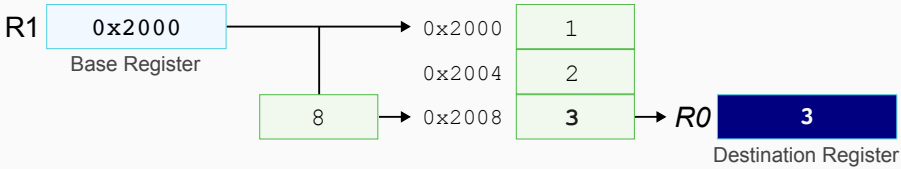
```
LDR   R0, [R1], #8
@ address = R1
@ R1 = R1 + 8 (after)
```

Read the brackets, not the mnemonic

- Offset inside the brackets → pre-indexed.
- Offset outside the brackets → post-indexed.
- Trailing ! → writeback.

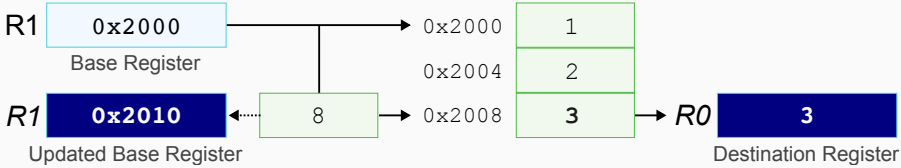
© Muním Zabidi

Visualizing Offset Access `LDR R0, [R1, #8]`



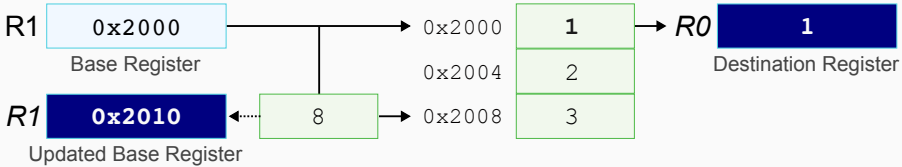
`R0` is loaded with the address pointed by `R1 + 8` — `0x2008`. `R1` is not updated.

Visualizing Pre-Indexed Access `STR R0, [R1, #16]!`



The address is computed *first*: `0x2000 + 16 = 0x2010`. This is both where `R0` is loaded from, and where `R1` ends up. One calculation, used twice.

Visualizing Post-Indexed Access `STR R0, [R1], #16`



`R0` is loaded from address `R1` already held — `0x2000` — *before* anything changes. `R1` is updated *afterward*, to `0x2010`, ready for the next access.

These Modes Are Not Just for Arrays

Offset, pre-indexed, and post-indexed addressing are useful for any pointer, not only array indices:

- A structure field at a fixed offset from a base pointer — `[R1, #4]`
- A buffer walked one element at a time, letting the pointer advance itself — `[R1], #4`
- A local variable in a stack frame at a fixed offset from the frame pointer — `[R7, #8]`

Every one of these uses a constant offset. No second register or shift is needed. The next section adds a register offset—and then a scaled register offset—when the offset itself is data, such as a run-time index.

Dynamic Indexed Access

Form	Written	Typical C equivalent
Immediate offset	`[R1, #4]`	`*(p + 1)`
Register offset	`[R1, R2]`	`*((char *)p + offset)`
Scaled register offset	`[R1, R2, LSL #2]`	`p[i]`

The key idea

A register offset supplies a displacement at run time. A scaled register offset also multiplies that displacement by a power of two, which makes it ideal for array indexing.

For a 32-bit element:

$$\text{address} = \text{base} + (\text{index} \times 4)$$

Scaled Register Offset

```
LDR   R0, [R1, R2, LSL #2]
```

- R1 — base address of the array
- R2 — the index
- LSL #2 — scale by 4, the element size

An immediate offset is fixed when you write the program; a register offset can change while the program runs, which is exactly what a loop counter needs.

Indexing without a multiply

The barrel shifter feeds directly into address generation, so scaling the index, adding the base, and loading the value all happen in one instruction. No separate shift or `MUL` is needed.

This is Chapter 3's formula, `base + (index * element size)`, computed by hardware. It is why that multiplication never appears explicitly in compiler output.

Shift	Scales by, matching
LSL #1	2 — a 16-bit halfword (`int16_t`)
LSL #2	4 — a 32-bit word (`int32_t`, any pointer)

Array Indexing Becomes a Scaled Offset

C

```
int x = array[i];
```

For a 32-bit `int`, the compiler computes the address as $\texttt{array} + i \times 4$.

C type	Typical scaling	
`int16_t *`	`LSL #1`	(2 bytes)
`int32_t *`	`LSL #2`	(4 bytes)
`uint32_t *`	`LSL #2`	(4 bytes)

Typical ARM

```
@ R1 = array
@ R2 = i
LDR   R0, [R1, R2, LSL #2]
```

Address calculation

$$R1 + (R2 \times 4)$$

The shift scales the index before it is added to the base address.

This is why array access on ARM is so cheap. If someone says “the barrel shifter is just a historical curiosity,” they have not looked at what the compiler produces for an array loop.

Array Indexing vs Pointer Arithmetic

Same address, different notation

```
int x = array[i];
int x = *(array + i);

int *p = array;
int x = p[i];   // same as *(p + i)
```

In every case, `i` is an *element index*. Pointer arithmetic automatically accounts for the size of the pointed-to type.

What changes at the machine level?

The compiler turns the element index into a byte displacement:

$$i \times \texttt{sizeof(int)}$$

A useful distinction

`p + i` advances by `i` *elements*.

`(char *)p + i` advances by `i` *bytes*.

Worked Example: Array Processing

C code

```c
int array[10];
int sum = 0;
int i;

for (i = 0; i < 10; i++) {
        sum += array[i];
}
```

ARM assembly

```
    LDR   R1, =array  @ base address
    MOV   R0, #0      @ sum = 0
    MOV   R2, #0      @ i = 0
loop:
    LDR   R3, [R1, R2, LSL #2]
    ADD   R0, R0, R3  @ sum += a[i]
    ADD   R2, R2, #1  @ i++
    CMP   R2, #10
    BLT   loop
```

This example combines

- `LDR =` to load an address
- a scaled register offset for the index
- register-only arithmetic
- `CMP` and a conditional branch for the loop

The `LSL #2` handles the array arithmetic; no multiply instruction appears.

© Munim Zabidi

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. Shift Basics
5. The Barrel Shifter
6. Addressing Modes
7. **Advanced Arithmetic Operations**
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. Summary

Advanced Arithmetic: Carry and NOP Two more instructions worth knowing

ADC and SBC

- `ADC Rd, Rn, Op2` : `Rd = Rn + Op2 + C`
- `SBC Rd, Rn, Op2` : `Rd = Rn − Op2 − C̄`
- For arithmetic wider than one register: `uint64_t`, or a bignum.
- Add the low words with `ADDS` to capture the carry. Then add the high words with `ADC`.

```
uint64_t sum = a + b;
```

```
ADDS  R0, R0, R2  @ low words, sets C
ADC   R1, R1, R3  @ high words + carry
```

NOP

- Changes no register, no memory, no flag.
- Advances the PC and uses one cycle.
- C has no operator for it. An empty statement `;` may be deleted by the compiler. A `NOP` may not.
- Used to pad a short delay, to align code, or to leave space for a patch.

Already covered

`RSB` (negation) and `SDIV`/`UDIV` (2–12 cycles, no flags) are in the arithmetic table at the start of the chapter.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Arithmetic Instructions

3 Logical Instructions and Bit Manipulation

4 Shift Basics

5 The Barrel Shifter

6 Addressing Modes

7 Advanced Arithmetic Operations

8 **Optional: How a Compiler Chooses Between Shift and Multiply**

9 Summary

Optional: Shift or Multiply? Three Constants One C line, three different strategies

The function `return raw * K;` compiles differently for each constant `K`. The argument arrives in `R0` and the result leaves in `R0`.

Constant	Instruction	Why
$\times 5$	`ADD R0, R0, R0, LSL #2`	$5 = 4 + 1$: the shifter makes $4\times$, the ALU adds the original
$\times 8$	`LSL R0, R0, #3`	a pure shift; no addition needed
$\times 7$	`RSB R0, R0, R0, LSL #3`	$7 = 8 - 1$: subtract, with `RSB` reversing the order
$\times 11$	`MOV R1, #11` then `MUL R0, R0, R1`	not a power of two, nor one away from one: a real multiply

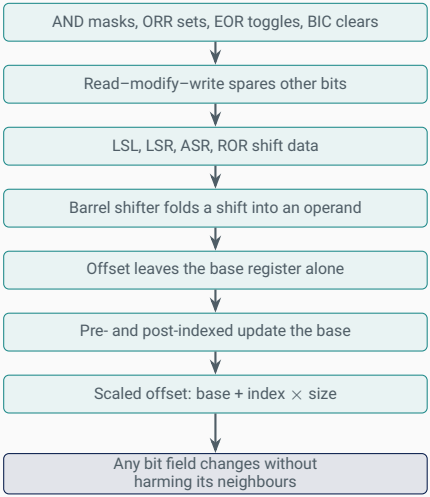
This is what “reading assembly” means

Not decoding a strange language. Recognising a few patterns, and asking why the compiler chose each one. This is not a rule: a different compiler or setting may choose differently.

Table of Contents

1. Why This Matters
2. Arithmetic Instructions
3. Logical Instructions and Bit Manipulation
4. Shift Basics
5. The Barrel Shifter
6. Addressing Modes
7. Advanced Arithmetic Operations
8. Optional: How a Compiler Chooses Between Shift and Multiply
9. **Summary**

Summary



Terminology covered

- AND, ORR, EOR, BIC, ORN
- Shifts and rotate
- Flexible second operand
- Pre- and post-indexed
- Scaled register offset

Next: Chapter 7

Control Flow, Functions and the Stack

We can move data. Next, we decide what to do with it.

© Muním Zabidi